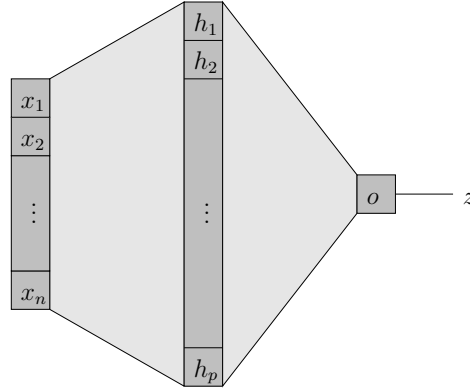


COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 6

1 Artificial Intelligence (sbh11)

A neural network takes input vectors $\mathbf{x} \in \mathbb{R}^n$, has a single layer of hidden nodes h_i where $i = 1, \dots, p$, and a single output node o .



All nodes compute the function $z = \sigma(a)$ where

$$a = \sum_{i=1}^m w_i z_i + w_0.$$

Here z and z_i denote the inputs and outputs of the node, and each node has its own set of weights $w_0, w_1, \dots, w_m$. Examples take the form $(\mathbf{x}, y)$ and the error the network makes for an example is $E(\mathbf{x}, y, \mathbf{w})$, where $\mathbf{w}$ is the collection of all the weights in the network.

Answer: Throughout the solution notes, numbers in brackets provide a *very approximate guide* to allocation of marks.

Backpropagation

- (a) An example has been applied to the network and we know the quantity $\delta = \partial E / \partial a$ for the output node o . Explain how this knowledge can be used to compute the partial derivative of E with respect to the weights for some hidden node h_i . [8 marks]

Answer: This is covered in detail during the lectures and is a subset of the derivation for the standard backpropagation procedure. If we are interested in $\mathbf{w}_i^{(j)}$ —the j th weight for node h_i —then as each hidden node is only connected to a single output node

$$\frac{\partial E}{\partial \mathbf{w}_i^{(j)}} = \frac{\partial E}{\partial a} \frac{\partial a}{\partial \mathbf{w}_i^{(j)}} = \delta \frac{\partial a}{\partial \mathbf{w}_i^{(j)}}$$

(1). Then

$$\frac{\partial a}{\partial \mathbf{w}_i^{(j)}} = \frac{\partial a}{\partial z_i} \frac{\partial z_i}{\partial a_i} \frac{\partial a_i}{\partial \mathbf{w}_i^{(j)}}$$

(3). Each of these derivatives is straightforward:

$$\begin{aligned}\frac{\partial a}{\partial z_i} &= w_i \\ \frac{\partial z_i}{\partial a_i} &= \sigma'(a_i) \\ \frac{\partial a_i}{\partial \mathbf{w}_i^{(j)}} &= x_j\end{aligned}$$

(1 each). Here w_i is the i th output node weight, and $x_0 = 1$ (1).

Training using
gradient descent

(b) The hidden nodes are now replaced with a different type of node, computing

$$z_i = h_i(\mathbf{x}) = \phi(\|\mathbf{x} - \mathbf{c}_i\|^2)$$

where ϕ is some new function, the $\mathbf{c}_i \in \mathbb{R}^n$ are the parameters for the new nodes, and

$$\|\mathbf{v}\|^2 = \mathbf{v}^T \mathbf{v}.$$

Give a detailed derivation of a training algorithm for this network. [12 marks]

Answer: This kind of node is not mentioned in the lectures, but the solution is a straightforward application of the idea of gradient descent. The solution should give the usual update equation for gradient descent (1) and derive an expression for the partial derivative of E with respect to each weight in the network. If an example has been applied to the network we proceed as follows. For the weights in o , as

$$o = \sigma \left(\sum_{i=1}^m w_i z_i + w_0 \right).$$

we have

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial o} \frac{\partial o}{\partial a} \frac{\partial a}{\partial w_i}$$

(3). The first term is obtained depending on the actual form of E (1), the second is $\sigma'(a)$ (1), and the third is z_i with $z_i = 1$ if $i = 0$ (1).

For a weight $\mathbf{c}_i^j$ we have

$$\frac{\partial E}{\partial \mathbf{c}_i^{(j)}} = \frac{\partial E}{\partial a} \frac{\partial a}{\partial \mathbf{c}_i^{(j)}}$$

(2). The first term is as above. The second is

$$\begin{aligned}\frac{\partial a}{\partial \mathbf{c}_i^{(j)}} &= \frac{\partial}{\partial \mathbf{c}_i^{(j)}} \left(\sum_{i=1}^p w_i \phi(\|\mathbf{x} - \mathbf{c}_i\|^2) + w_0 \right) \\ &= w_i \frac{\partial}{\partial \mathbf{c}_i^{(j)}} \phi(\|\mathbf{x} - \mathbf{c}_i\|^2) \\ &= w_i \phi'(\|\mathbf{x} - \mathbf{c}_i\|^2) \frac{\partial}{\partial \mathbf{c}_i^{(j)}} \|\mathbf{x} - \mathbf{c}_i\|^2 \\ &= -2w_i \phi'(\|\mathbf{x} - \mathbf{c}_i\|^2) (x_j - \mathbf{c}_i^{(j)})\end{aligned}$$

(1 each for the last three lines).
